

HeadQ: Model-Visible Distortion and Score-Space Correction for KV-Cache Quantization

Jorge L. Ruiz Williams
Independent Researcher
jorgeruizwilliams@gmail.com

May 6, 2026

Abstract

KV-cache quantizers usually optimize storage-space reconstruction, even though attention reads keys through logits and values through attention-weighted readout. We argue that persistent cache error should be measured in model-visible coordinates. For keys, the visible object is score error modulo constant shifts; this yields HeadQ, a key-side method that stores a low-rank residual side code in a calibration-learned query basis and applies it as an additive logit correction. For values, fixed-attention readout gives an A^2 -weighted token-distortion surrogate. Across six models, Fisher/score-space error predicts attention KL far better than raw key MSE; same-budget counterexamples, null-space interventions, query-PCA controls, and wrong-sign HeadQ falsify storage-MSE alternatives. Matched Pythia checkpoints localize the main anomaly to a small-model low-entropy route-flip boundary. In K-only WikiText-103 decode experiments with dense values, HeadQ removes roughly 84–94% of the excess perplexity on the strongest 2-bit rows; in an auxiliary full-KV 2-bit composition, HeadQ plus an A^2 value policy improves all six models.

1 Introduction

KV caches are the persistent memory of autoregressive transformers [1, 2]. During decoding, every new token reads previous keys and values, so cache size and bandwidth grow with context length [9, 10, 11]. Recent systems papers already exploit practical K/V asymmetries [9, 15, 16, 17], but the dominant optimization targets are still mostly storage-centered: mean-squared error, distortion, outlier suppression, or rotation quality [26, 14, 13, 22]. That is convenient, but it is not what attention reads.

This paper has one central claim: persistent cache error should be measured in model-visible coordinates. For keys, the relevant object is logit error

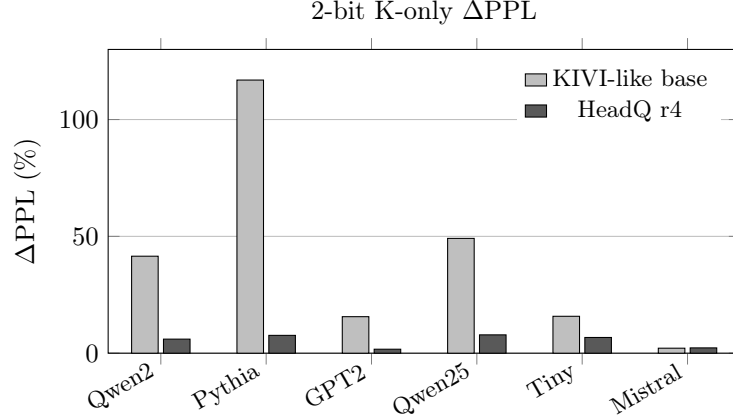


Figure 1: Headline key-side payoff on the damage-bearing 2-bit K-only rows. HeadQ lowers ΔPPL relative to the KIVI-like baseline across the main affected models; lower is better.

modulo constant shifts, because a key affects the next token only through query-key scores. For values, the relevant object is attention-weighted readout error, because values are consumed only through $\sum_j p_j v_j$. HeadQ is the key-side method instance of this claim: it keeps a standard low-bit key baseline, stores a small residual side code in a calibration-learned query basis, and lets the read-time query turn that code into an additive logit correction. We use *score-visible* for key perturbations that change logits beyond an additive constant, *softmax-null* for perturbations that attention cancels, *route flip* for changes in the top attended token, and *readout-visible* for value perturbations that change the weighted output.

The evidence chain is deliberately compact. The fixed-query derivation identifies the score-visible quotient of key error. Fisher/KL ranking, paired MSE counterexamples, and a softmax-null mean intervention test whether storage MSE can fail behaviorally. Query-PCA then asks whether the score-active residual is learnable without future queries. Downstream decode experiments test HeadQ directly, while a small full-KV composition check uses the value-side A^2 surrogate suggested by the same operator-matched view. The scope remains narrow: the main PPL experiments are K-only with dense values, and the auxiliary full-KV rows are not a packed-kernel or latency claim.

The contributions are:

1. We formulate model-visible KV-cache distortion: score-visible key error for K, and readout-visible value error with an A^2 diagonal surrogate for V. This explains why raw key MSE and raw value MSE need different proxies.
2. We introduce HeadQ, a cache-write/read algorithm that stores a low-rank

key residual side code in a calibration-learned query basis and applies it directly as a logit correction.

3. We build a falsifiable evidence chain for that design: score-space predictors beat key MSE, paired 3-bit counterexamples and null-space interventions expose the MSE failure mode, deployable query-PCA bases recover the useful low-rank structure, same-budget wrong-sign controls show that deployed gains depend on correcting the right score object in the right direction, and matched Pythia checkpoints localize the remaining anomaly to a family-specific route-flip boundary case.

2 Experimental Setup

We study real-corpus decode-time key-cache perturbations on WikiText-103 [8]. At a held-out decode step, we replace cached keys by quantized–dequantized keys inside the forward pass and record the resulting score, attention, and output deltas. The main mechanism panel contains GPT-2 124M [1], Pythia-160M [6], Qwen2-0.5B [2], Qwen2.5-0.5B [3], TinyLlama 1.1B [7], and Mistral 7B [5]. Let q be the current query for one attention head, and let $K = [k_1, \dots, k_n]^\top \in \mathbb{R}^{n \times d}$ and $V = [v_1, \dots, v_n]^\top \in \mathbb{R}^{n \times d}$ be the cached keys and values. Dense attention computes

$$s_j = q^\top k_j / \sqrt{d}, \quad p = \text{softmax}(s), \quad o = \sum_{j=1}^n p_j v_j. \quad (1)$$

Quantized caches are written as

$$\hat{K} = K + E_K, \quad \hat{V} = V + E_V. \quad (2)$$

We measure key-side behavioral damage using attention KL, attention ℓ_1 drift, attention-output MSE, and output cosine. For a sampled layer, head, window, and decode position, key MSE is $\|\hat{K} - K\|_F^2 / (nd)$, score error is $\delta s_j = q^\top (\hat{k}_j - k_j) / \sqrt{d}$, attention KL is $\sum_j p_j \log(p_j / \hat{p}_j)$ with $\hat{p} = \text{softmax}(s + \delta s)$, and attention-output MSE is $\|\hat{o} - o\|_2^2 / d$ using the same dense values unless a value-side experiment is explicitly named. Fisher error is the local score quadratic in Eq. 4; table captions state when its square root or a ratio to a base quantizer is reported. Queries are not persistent cache entries, and value-side results enter only as boundary conditions for the broader K/V asymmetry. Unless otherwise noted, all experiments were run on a single laptop RTX 4090 GPU using PyTorch 2.5.1+cu124 and Hugging Face Transformers 5.2.0. The paper tables use Python-side eager-attention hooks for calibration and evaluation rather than custom CUDA kernels.

All paired metric comparisons use identical sampled sites within a model. For Table 1, each reported coefficient is a point-estimate within-model Spearman ρ_{KL} over matched quantizer candidates evaluated on the same sampled layer/head/window/position set. Thus the only quantity changing across the

predictor columns is the damage proxy, not the data, model, or evaluated cache sites.

We use two experiment classes. Oracle active-axis experiments access the current decode query and ask whether a one-dimensional score-relevant correction is sufficient to rescue behavior. They are diagnostic upper bounds, not cache formats. Deployable experiments replace the current query by a fixed per-head query-PCA basis learned from calibration queries and quantize residual coordinates in that basis; deployed negative controls either reverse the recovered score correction or replace the learned basis by an equal-budget random basis. The six-model non-oracle universality sweep behind Table 3 uses 12 WikiText-103 calibration prompts, 4 held-out evaluation prompts, target length 512, and 64 sampled heads per prompt. The downstream HeadQ frontier uses the dense prefill queries of each evaluation window itself as calibration data; with four WikiText-103 test windows at length 128 and prefill 64, each window contributes 64 post-RoPE query rows per head to its local basis fit before any optional row subsampling. Unless otherwise noted, the main downstream PPL experiments quantize keys only and keep values dense. Those results measure key-side damage and rescue rather than full-KV compression; the auxiliary full-KV composition check is explicitly labeled. Not every auxiliary probe spans exactly the same model set: the six-model paper panel is used for the point-estimate KL-ranking table, the 12-calibration-prompt non-oracle universality sweep, and the HeadQ frontier; the inert-mean control sweep covers GPT-2, Pythia, Qwen2, Qwen2.5, and TinyLlama plus auxiliary smaller models, but not Mistral; the oracle active-axis ceiling is reported both as a four-model average and as a Pythia-focused high-damage slice.

3 Model-Visible KV Geometry

3.1 Key Score Space and Softmax Fisher Geometry

A key perturbation δk_j induces score perturbation

$$\delta s_j = q^\top \delta k_j / \sqrt{d}. \quad (3)$$

The local second-order geometry of the softmax distribution is

$$\mathcal{I}_{q,K}(\delta K) = \delta s^\top (\text{diag}(p) - pp^\top) \delta s, \quad (4)$$

where $p = \text{softmax}(s)$ and $\text{diag}(p) - pp^\top$ is the softmax Fisher information matrix [19]. More precisely, if $\hat{p} = \text{softmax}(s + \delta s)$, then

$$\text{KL}(p \parallel \hat{p}) = \frac{1}{2} \mathcal{I}_{q,K}(\delta K) + O(\|\delta s\|_2^3) \quad (5)$$

for small score perturbations. We therefore use Eq. 4 as a local diagnostic and ranking proxy for attention damage, not as a global equality for downstream loss. Recent work on exact attention sensitivity likewise studies softmax sensitivity through attention-Jacobian norms rather than raw storage distance [20].

Importantly, Eq. 4 does not require a deployed method to form or optimize a dense Fisher matrix. It is exactly the p -weighted centered variance of score error:

$$\mathcal{I}_{q,K}(\delta K) = \sum_{j=1}^n p_j \left(\delta s_j - \sum_{i=1}^n p_i \delta s_i \right)^2. \quad (6)$$

Thus the Fisher object is a derivation and diagnostic; the computational primitive is cheaper: compute score errors, remove the softmax-inert constant, and preserve the remaining score-visible coordinates. Dropping or approximating the p weights keeps the same quotient nullspace. A write-time approximation can replace the future query q by a calibration-learned per-head query basis U_h and minimize

$$D_{U_h}(\hat{K}, K) = \left\| C \left((\hat{K} - K) U_h \right) \right\|_F^2, \quad (7)$$

where C removes token-shared components across cache positions. This reaches the Fisher quotient target through low-rank projections, centering, and ordinary quantizer fitting rather than online Fisher optimization.

The key geometry also contains an exact null direction. If every cached key is translated by the same vector μ , then for any query q ,

$$q^\top (k_j + \mu) = q^\top k_j + q^\top \mu, \quad (8)$$

and the added term is the same for every token index j . Therefore

$$\text{softmax}(q(K + \mathbf{1}\mu^\top)^\top / \sqrt{d}) = \text{softmax}(qK^\top / \sqrt{d}). \quad (9)$$

Thus a token-shared key translation is softmax-null for any fixed query. More generally, if $\hat{q} = q/\|q\|_2$ and we decompose each key perturbation row as

$$\delta k_j = \bar{\delta} + a_j \hat{q} + \nu_j, \quad q^\top \nu_j = 0, \quad (10)$$

then the induced score perturbation is

$$\delta s_j = q^\top \bar{\delta} / \sqrt{d} + \|q\|_2 a_j / \sqrt{d}. \quad (11)$$

The first term is constant across tokens, so it is softmax-null; the orthogonal term ν_j is invisible to the current query. Attention therefore depends only on the centered active coefficients a_j . Storage MSE, by contrast, pays for all three pieces: the active coefficients, the token-shared shift, and the orthogonal residual. This is the core mismatch behind the paper: a perturbation can have large storage error and zero attention damage, while a smaller perturbation in the active direction can change attention sharply. Appendix A gives the formal quotient-space statement and proofs.

This decomposition explains why a fixed grid can waste resolution on key components that are large in storage coordinates but invisible to dense attention. Mean-key removal is one simple example of choosing a better representative. The broader lesson is practical: key quantization should preserve score-visible coordinates, not just raw Euclidean reconstruction.

The decomposition gives three empirical predictions: key MSE should be a weak behavioral metric, score-space error should rank attention damage better, and non-equivariant quantizers should fail by converting benign key components into active score error. It also gives a method: preserve a compact approximation to the score-visible residual and apply it where scores are computed.

3.2 Value Readout Geometry

Values are consumed through readout, not logits. For fixed attention weights p , a value perturbation induces

$$\Delta o = \sum_{j=1}^n p_j \Delta v_j, \quad \frac{1}{d} \|\Delta o\|_2^2 = \frac{1}{d} \left\| \sum_{j=1}^n p_j \Delta v_j \right\|_2^2. \quad (12)$$

If cross-token value residuals are weakly correlated, the cross terms average out and the diagonal surrogate becomes

$$\frac{1}{d} \|\Delta o\|_2^2 \approx \sum_{j=1}^n p_j^2 \frac{\|\Delta v_j\|_2^2}{d}. \quad (13)$$

Thus the value-side analogue of key Fisher is not raw value MSE but an A^2 -weighted token distortion. This gives a concrete reason for the K/V asymmetry emphasized by KIVI and later key-favored allocators [9, 15]: K and V are read through different operators. An auxiliary Qwen2 decode-head value probe confirms the ranking consequence: the Spearman correlation to exact output MSE is 0.984 for A^2 -weighted token MSE, versus 0.746 for top-1 token error, 0.636 for A -weighted token error, and 0.492 for raw value MSE.

4 HeadQ: Key-Side Method

HeadQ implements the score-space correction suggested by the fixed-query decomposition. It is a key-side side-code method layered on top of an ordinary low-bit key quantizer. At cache write time, HeadQ stores the base quantized key and a low-rank code for the base residual in a fixed per-head query basis learned from calibration queries. At attention read time, the current query converts that code into an additive correction to the logits. Values are unchanged in the K-only experiments.

For each layer/head h , calibration collects post-RoPE queries and stores the top- r PCA directions as $U_h \in \mathbb{R}^{d \times r}$ with orthonormal columns. Given a base key quantizer B_b at cache write time,

$$\hat{k}_j^{\text{base}} = B_b(k_j), \quad (14)$$

$$e_j = k_j - \hat{k}_j^{\text{base}}, \quad (15)$$

$$z_j = Q_q(U_h^\top e_j), \quad (16)$$

Algorithm 1: HeadQ key-side logit correction

Inputs: base key quantizer B_b , side quantizer Q_q , rank r , calibration queries, stream of keys k_j , read-time query q .

Calibration: for each layer/head h , collect post-RoPE calibration queries and store the top- r PCA basis $U_h \in \mathbb{R}^{d \times r}$.

1. **Cache write for key k_j :** store the base key $\hat{k}_j^{\text{base}} = B_b(k_j)$.
2. Compute the base residual $e_j = k_j - \hat{k}_j^{\text{base}}$.
3. Store the side code $z_j = Q_q(U_h^\top e_j)$.
4. **Attention read for query q :** dequantize z_j to $\tilde{z}_j$ and form logits with Eq. 17.
5. Apply the usual softmax to the corrected logits and read dense values in the K-only experiments.

Output: corrected key-side logits. HeadQ adds rq side bits per cached key/head before packing metadata.

where Q_q is the q -bit side quantizer. At read time, HeadQ dequantizes $\tilde{z}_j$ and uses

$$\hat{s}_j = q^\top \hat{k}_j^{\text{base}} / \sqrt{d} + (q^\top U_h \tilde{z}_j) / \sqrt{d}. \quad (17)$$

The side code stores rq bits per cached key/head before packing metadata. Reported side-overhead ratios use FP4 key storage as the denominator, $rq/(4d)$, matching the calibration artifacts. The method stores no current or future decode query and does not modify value cache entries.

5 Empirical Results

The experiments move from metric prediction to deployed behavior. Score-space metrics should rank attention damage, softmax-null interventions should expose storage-coordinate failure, query-PCA should approximate the active residual without future queries, and HeadQ should improve downstream decode while composing with the value-side A^2 objective from Section 3.2. The appendix keeps the full numeric grids and oracle ceiling.

5.1 Score-Visible Error Predicts Attention Damage

If keys are score objects, Fisher/score geometry should rank attention damage better than Euclidean key MSE. Table 1 gives the six-model comparison: Fisher wins on every model, with gains of roughly 0.62–0.69 Spearman points over raw key MSE. A separate four-model selector probe and two 3-bit TurboQuant-like depth probes recover the same ordering inside fixed candidate families, so this is not only a pooled-across-methods effect.

Pythia-160M is the only visibly weaker row, but the anomaly is still attention-side by construction: the coefficient is against attention KL on

Table 1: Exact Fisher-versus-MSE KL-ranking coefficients for the six-model panel. Fisher/score-space error ranks attention KL far better than raw key MSE on every model.

Model	Fisher	MSE	Gain
GPT-2 124M	0.995	0.329	+0.666
Pythia-160M	0.696	0.031	+0.665
Qwen2-0.5B	0.993	0.354	+0.639
Qwen2.5-0.5B	0.991	0.343	+0.648
TinyLlama 1.1B	0.996	0.302	+0.694
Mistral 7B	0.997	0.376	+0.621

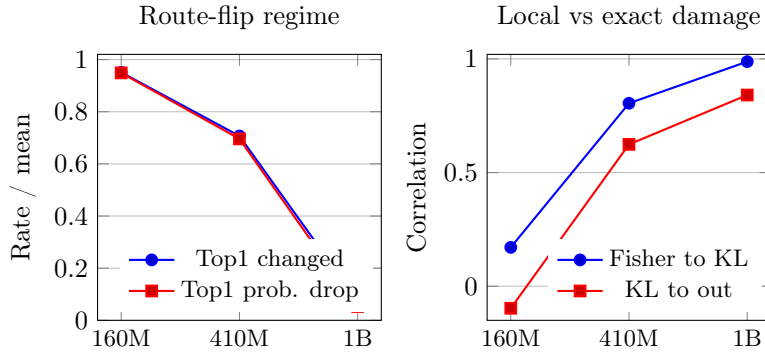


Figure 2: Matched low-entropy Pythia probes. The 160M anomaly is tied to an exact route-flip regime; the boundary weakens sharply by 410M and 1B within the same family.

matched cache sites, not against downstream loss. Auxiliary matched routing-regime probes sharpen the reason. The weak global coefficient comes from a narrow low-entropy slice under rotated/product 3-bit quantizers where finite perturbations frequently flip the top attended token; in that slice, exact reference-top-1 probability drop ranks KL much better than the local Fisher quadratic. The effect also weakens sharply across the matched Pythia checkpoints. The right takeaway is therefore narrow: not a general scaling law, but a family-specific suggestion that the low-entropy route-flip tail is concentrated at the small-model end of Pythia.

Fisher need not be deployed as a dense matrix computation. Eq. 6 gives an exactly equivalent score-variance form: compute score errors, subtract the attention-weighted constant mode, and measure the remaining weighted variance. In the four-model selector family, Fisher-style selection has held-out-KL Spearman 0.653, 0.794, 0.849, and 0.823 on GPT-2, Qwen2, Qwen2.5, and TinyLlama, versus 0.588, 0.272, 0.309, and 0.203 for key-MSE selection.

Table 2: K-side counterexample to raw key MSE. Lower key MSE does not imply lower KL or output error. Bold marks the better value within each model/metric comparison.

Model	Method	Key MSE	Fisher RMSE	Attn KL	Out MSE
Qwen2-0.5B	FPQ	0.137461	0.20481	0.05222	0.002205
Qwen2-0.5B	Ctrd. TQ-MSE	0.081915	0.46641	0.40414	0.006363
GPT-2 124M	FPQ	0.131712	0.17289	0.03480	0.002052
GPT-2 124M	Ctrd. TQ-MSE	0.040432	0.21239	0.05914	0.003535

A sharper counterexample compares a Fisher-pullback quantizer against a strong MSE-oriented vector-quantization baseline. The rows below are paired candidate means from the same four WikiText-103 windows (max length/stride 96/96, prefill 48): within each model, both quantizers are evaluated on the same dataset windows, layers, heads, cache positions, dense values, and average key budget. The Fisher-pullback quantizer uses a query-PCA axis chart with a 2/4-bit Fisher-selected coordinate budget at 3 average bits. The centered TurboQuant-MSE baseline subtracts the token-axis mean and applies a rotated MSE-oriented 3-bit quantizer with Gaussian Lloyd–Max scalar codebooks. On Qwen2-0.5B and GPT-2, centered TurboQuant-MSE has lower raw key MSE than Fisher-pullback FPQ, yet worse Fisher error, attention KL, and output MSE. Euclidean reconstruction improves while behavior gets worse.

Eq. 9 and the appendix derivation explain why these counterexamples occur. For a fixed query, token-shared and query-orthogonal key components can dominate Euclidean reconstruction error without affecting the current attention distribution. A Euclidean quantizer can therefore improve raw key MSE while still worsening the score errors that matter downstream.

This is the first necessity check, not only a prediction check: when a same-budget quantizer improves storage MSE by sacrificing the score-visible residual, attention KL and output error get worse. The following controls test the same point from the other direction. A softmax-null mean component is harmless to exact dense attention but can damage fixed-grid quantizers when it changes the active score grid; learned query bases rescue Fisher/KL damage, while equal-overhead random bases do not. Thus the metric is not just descriptive: damaging it, or failing to preserve its low-rank active coordinates, breaks the behavior the method is trying to protect.

5.2 Centering-Sensitive Quantizer Failure

The theorem predicts a concrete failure mode for low-bit grids. A storage-coordinate quantizer can fail in two steps: a token-shared common component changes the quantization grid, and that changed grid inflates error in the one-dimensional active coefficients that control scores. This separates a harm-

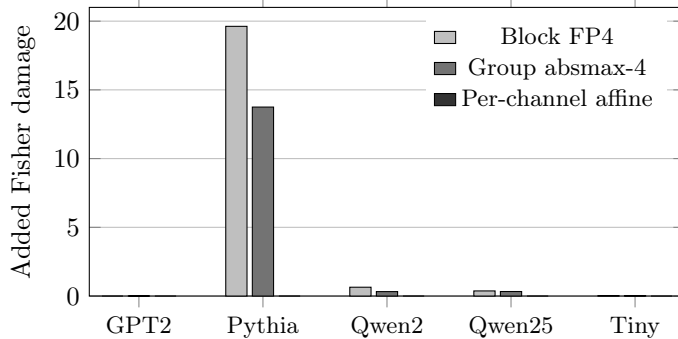


Figure 3: Softmax-null mean-key intervention at $\alpha = 1$. Restoring an attention-inert common key component sharply increases Fisher damage for non-equivariant fixed-grid K quantizers, while the per-channel affine control remains invariant.

less reparameterization from a quantization failure: exact attention cancels the common component, but the quantizer may not. We isolate this with an α intervention,

$$K_\alpha = \Delta K + \alpha \mathbf{1} \bar{k}^\top, \quad (18)$$

that holds the residual ΔK fixed while restoring the token-shared mean component $\bar{k}$. Exact attention is unchanged for every α , but fixed-grid and group quantizers are not translation-equivariant. The $\alpha = 1$ slice is easiest to read visually: Pythia is the sharpest example, but Qwen2, Qwen2.5, and TinyLlama show the same direction at smaller scale. Appendix Table 4 retains the full Fisher/KL control sweep.

The intervention separates coordinate description from quantized behavior. The dense model is unchanged; only the quantized pipeline changes. KIVI-like per-channel keys avoid this null-range waste by construction [9], and KVQuant-style centering removes the offending component before a fixed grid sees it [10]. The common key component is not Fisher-active by itself; damage appears only when a non-equivariant quantizer lets invisible storage energy set the active score grid.

5.3 Calibration-Learned Query Bases

Appendix B reports the oracle active-axis ceiling, which is useful only as a diagnostic upper bound: if the current query were known, one query-active scalar per token and head nearly removes Fisher/KL damage on Pythia-160M even while whole-key MSE stays near 0.98. The deployable question is whether active directions concentrate in a learnable per-head subspace. A six-model universality sweep with 12 calibration prompts, 4 evaluation prompts, ranks 1/2/4/8, and equal-overhead random controls answers yes for the centered and affine bases.

Table 3: Deployable query-PCA approximation from the six-model, 12-calibration-prompt sweep. Learned score-visible bases recover most of the diagnostic oracle benefit, while equal-overhead random bases do not.

Basis	F/Base	KL/Base	MSE/Base	Axis cap.	Over-head
Ctrd. Q-PCA r1 q4	0.2801	0.5240	0.9857	0.8144	0.0143
Ctrd. Q-PCA r4 q4	0.1727	0.1920	0.9416	0.8718	0.0573
Ctrd. Q-PCA r8 q4	0.1463	0.1586	0.8880	0.8967	0.1146
Affine Q-PCA r8 q4	0.1396	0.1463	0.8762	0.8967	0.1146
Ctrd. random r1 q4	1.0961	1.5035	0.9861	0.0151	0.0143

Table 3 reports the stable paper-facing rows from that sweep: centered block-FP4 rank-8 reaches pooled Fisher/Base 0.146 and KL/Base 0.159; per-channel affine rank-8 reaches Fisher/Base 0.140 and KL/Base 0.146. Query-axis capture rises to 0.897, while equal-overhead random bases fail.

5.4 Downstream HeadQ Evaluation

The main downstream evidence is concentrated on the rows where the baseline actually breaks. With dense values and a KIVI-like per-channel affine key baseline [9], HeadQ is strongest on Qwen2, Pythia, GPT-2, and Qwen2.5, where rank-4 HeadQ removes roughly 84–94% of the excess PPL. TinyLlama also improves materially, while Mistral is already near dense in this protocol. Appendix Table 5 retains the full 2/3/4-bit frontier; near-dense rows there should be read as noise-floor checks rather than monotone-gain tests.

A same-budget sign control checks that the gain comes from correcting the score-visible residual in the right direction, not merely from adding a side stream. Reversing the HeadQ logit patch removes most of the 3-bit Qwen2 rescue and becomes catastrophically bad on Pythia-160M; Appendix Table 6 keeps the exact numbers. The main visual emphasis is instead the full-KV 2-bit composition, where the key-side correction and A^2 value policy improve every damage-bearing row.

As an auxiliary boundary check, we also compose the key-side HeadQ correction with the value-side A^2 allocator from Section 3.2. At 2 bits, the composed codec improves all six models. Appendix Table 7 reports the same four-window WikiText-103 protocol across 2/3/4 bits. This is still not a serving-kernel claim; it is evidence that the key-side correction composes naturally with the value-side objective.

Together, the results support the paper’s narrower method claim: key residuals should be compressed in score-visible coordinates, HeadQ is a practical approximation to that object, and the value side follows a different A^2 -weighted readout geometry. The 2-bit downstream figures carry most of the evidence; near-dense rows and the auxiliary full-KV check should not be overclaimed.

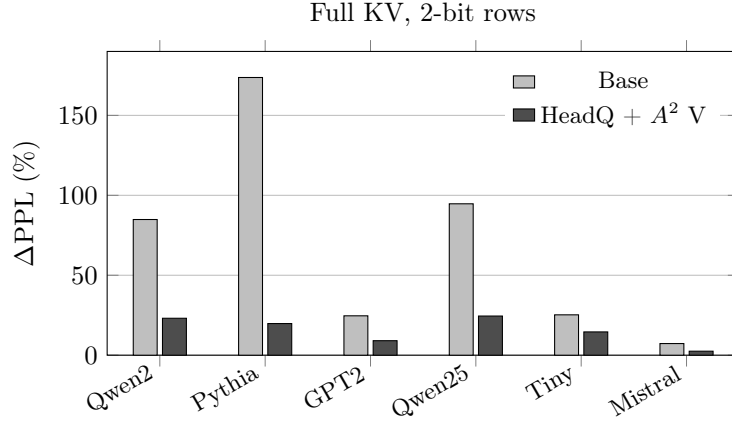


Figure 4: Full-KV auxiliary composition on the damage-bearing 2-bit rows; lower is better. The key-side HeadQ correction composes with the A^2 -guided value policy and improves all six models. Appendix Table 7 retains the 2/3/4-bit numeric grid.

6 Discussion

The design lesson is operator matching. Keys are score objects, so good key compression avoids softmax-null range waste, protects query-visible residuals, and applies corrections where scores are computed. Values are readout objects, so the natural first surrogate is A^2 -weighted token error. This view helps explain why KIVI-like per-channel keys, KVQuant-style centering, key-favored precision schedules, and value readout weighting can all help for different reasons [9, 10, 15].

The tests cover different failure modes of storage MSE. Fisher-versus-MSE predicts attention KL; the paired counterexample reverses the storage ranking; the softmax-null intervention changes quantized behavior without changing dense attention; query-PCA makes the active residual deployable; and the wrong-sign control verifies the direction of the deployed patch. The Pythia anomaly then becomes a boundary case: ultra-sharp attention rows under rotated/product 3-bit perturbations enter a finite route-flip regime, while matched 410M and 1B checkpoints recover the local ranking.

7 Relation to KV-Cache Quantization Work

KV-cache quantization methods such as KIVI [9], KVQuant [10], GEAR [11], QuaRot-style rotations [13], AXELRAM [21], RotateKV [22], PolyKV [23], LOOKAT [24], and SpinQuant [25] reduce cache memory through practical serving designs. Token-importance methods such as H2O [12] choose which cache entries to retain or protect. Generic vector quantization and rotation meth-

ods, including TurboQuant-style approaches [14] and product quantization [26], optimize storage-centered objectives such as MSE or product-code distortion.

The closest papers are asymmetry-oriented. Quantize What Counts [15] argues for key-favored precision using spectral and Frobenius structure; AsymKV [16] and KVTuner [17] use layer-wise sensitivity. Our claim is narrower: key sensitivity should be measured in query-conditioned behavioral geometry, not only by layer-level or norm-level statistics.

KIVI [9] is conceptually important because it discovers a granularity asymmetry: per-channel keys and per-token values. Our fixed-query view gives one key-side reason such asymmetries can arise. TriAttention [18] is especially relevant geometrically: it uses pre-RoPE Q/K centers to estimate key importance for long-reasoning KV compression, whereas our object is a post-RoPE score-space mean-key component whose logit contribution is constant across positions and therefore softmax-inert. KIVI-like per-channel K repairs bad key coordinates; KVQuant-style centering avoids softmax-null range waste; TurboQuant-style MSE can be strong for value-side readout reconstruction while missing key-side quotient geometry. The central distinction is storage-coordinate versus model-visible distortion.

8 Limitations

The paper is mechanistic, not a serving-system claim. Oracle active-axis probes are diagnostic upper bounds; the main downstream PPL rows quantize keys and keep values dense; the auxiliary full-KV composition is unoptimized and includes side-code overhead before any packed metadata design. We do not study packed kernels, latency, or throughput.

The model panel uses short real-corpus windows and six models up to Mistral 7B, below the 7B–70B frontier of recent systems papers [15, 16, 17]. The six-model KL-ranking table is a point-estimate summary rather than a confidence-interval result, and not every auxiliary probe spans exactly the same model set. The matched Pythia probes explain one verified family-specific route-flip boundary rather than a universal scaling law.

The deployed wrong-sign control tests correction direction but does not exhaust all same-budget side-code alternatives. Random side bases can be model- and window-dependent, so we use them only as attention-level controls. Query quantization also remains out of scope: queries are not persistent cache entries, and we do not show that the key-side chart transfers cleanly to Q.

9 Conclusion

KV-cache distortion should be defined in the coordinates the model reads. For keys, that is score error modulo constant shifts; for values, it is attention-weighted readout error with an A^2 diagonal surrogate. HeadQ is the key-side consequence: store a small residual code in a calibration-learned query basis

and apply it directly as a logit correction.

The evidence supports that operator-matched view. Score-space metrics beat raw MSE, paired controls expose storage-coordinate failures, query-PCA makes the useful residual deployable, and full-KV 2-bit composition shows the key correction can coexist with the value objective. The next step is to complete that same story as an optimized full-KV codec rather than treating storage MSE as the default target.

Appendix

A Formal Statements and Proofs

This appendix collects formal statements, proofs, and diagnostic upper-bound details that would otherwise interrupt the main method story.

Proposition 1 (Softmax-invariant key translation). *For any query q and any token-shared key translation μ , replacing each cached key k_j by $k_j + \mu$ leaves the dense attention distribution unchanged.*

Proof. The replacement adds $q^\top \mu / \sqrt{d}$ to every score. Softmax is invariant to adding a constant multiple of $\mathbf{1}$ to its input, so the attention probabilities are unchanged. $\square$

Theorem 1 (Fixed-query representation of visible key error). *Fix a nonzero query q . Define the null set*

$$\mathcal{N}_q = \{ \mathbf{1}\mu^\top + N : \mu \in \mathbb{R}^d, q^\top n_j = 0 \text{ for every row } n_j \text{ of } N \}. \quad (19)$$

Two key perturbations δK and $\delta K'$ induce the same attention distribution for every base score vector whenever $\delta K - \delta K' \in \mathcal{N}_q$. Equivalently, the behaviorally visible perturbation space for keys at query q is the quotient

$$(\mathbb{R}^d)^n / \mathcal{N}_q \cong \mathbb{R}^n / \text{span}\{\mathbf{1}\}. \quad (20)$$

In coordinates, define $\hat{q} = q / \|q\|_2$. For any key perturbation δK with rows δk_j , let $\bar{\delta} = \frac{1}{n} \sum_{j=1}^n \delta k_j$ and decompose

$$\delta k_j = \bar{\delta} + a_j \hat{q} + \nu_j, \quad q^\top \nu_j = 0. \quad (21)$$

Then the score perturbation is

$$\delta s_j = q^\top \bar{\delta} / \sqrt{d} + \|q\|_2 a_j / \sqrt{d}. \quad (22)$$

Hence dense attention depends on δK only through the per-token active coefficients a_j , modulo additive constants. The token-shared shift $\bar{\delta}$ and the orthogonal components ν_j are softmax-null.

Proof. For any perturbation matrix δK , the induced score perturbation is the vector $\delta s = \delta K q / \sqrt{d}$. If $\delta K - \delta K' \in \mathcal{N}_q$, then $\delta s - \delta s' = c \mathbf{1}$ for some scalar c , so $\text{softmax}(s + \delta s) = \text{softmax}(s + \delta s')$ for every base score vector s . Conversely, if two perturbations differ by a score vector that is not constant, one can choose a base score vector for which softmax separates the two distributions; the visible score space is therefore $\mathbb{R}^n / \text{span}\{\mathbf{1}\}$. The coordinate formula follows by decomposing each row into its token mean, projection onto $\hat{q}$, and a component orthogonal to q . $\square$

Corollary 1 (Storage MSE does not order key behavior). *For a fixed query q , attention and the local softmax Fisher quadratic depend only on the centered active coefficients a_j from the theorem above. In contrast, storage MSE depends on all components of δK : the active coefficients, the token-shared component, and the per-token orthogonal components. Therefore there exist key perturbations with arbitrarily large storage MSE and exactly unchanged attention. Moreover, storage MSE cannot rank behavioral damage in general: a large null perturbation can have zero attention damage, while a smaller active perturbation can have nonzero Fisher and KL.*

Proof. Scale any nonzero element of $\mathcal{N}_q$ to make $\|\delta K\|_F^2/(nd)$ arbitrarily large while leaving attention unchanged by the theorem. A perturbation with a non-constant active coefficient vector changes scores modulo constants and therefore has nonzero Fisher quadratic for generic p . These two perturbations give the claimed ordering failure for storage MSE. $\square$

Proposition 2 (Fixed-attention value surrogate). *Fix attention weights p and value perturbations Δv_j . The exact readout error is*

$$\Delta o = \sum_{j=1}^n p_j \Delta v_j. \quad (23)$$

If the cross-token inner products $\Delta v_i^\top \Delta v_j$ average to zero for $i \neq j$, then

$$\mathbb{E} \frac{\|\Delta o\|_2^2}{d} = \sum_{j=1}^n p_j^2 \mathbb{E} \frac{\|\Delta v_j\|_2^2}{d}. \quad (24)$$

Thus A^2 -weighted token distortion is the diagonal value-side analogue of exact readout error.

Proof. Expand the squared norm:

$$\frac{\|\Delta o\|_2^2}{d} = \sum_{j=1}^n p_j^2 \frac{\|\Delta v_j\|_2^2}{d} + \sum_{i \neq j} p_i p_j \frac{\Delta v_i^\top \Delta v_j}{d}. \quad (25)$$

Under the stated assumption, the cross terms vanish in expectation, leaving the A^2 -weighted diagonal form. $\square$

B Oracle Ceiling and Full Numeric Tables

Oracle active-axis experiments are included only as a diagnostic upper bound. On Pythia-160M, preserving one query-active scalar per token and head drives Fisher/Base and KL/Base near zero while whole-key MSE stays near 0.98, indicating that the harmful key-side error is low-dimensional even when Euclidean reconstruction barely changes. The main paper therefore keeps only the deployable query-PCA sweep; the oracle ceiling lives here because it motivates the method but is not itself a cache format.

Table 4: Full causal null-space control at $\alpha = 1$. Restoring a softmax-null mean-key component increases damage for fixed-grid block and group absmax-4 K quantizers, while per-channel affine K remains translation-equivariant. Values near zero indicate no inert-mean penalty.

Model	Common E	Block F	Block KL	G4 F	G4 KL	Affine F
GPT-2 124M	0.4869	-0.0106	-0.0120	0.0067	0.0057	-0.0000
Pythia-160M	0.8762	19.6225	8.7749	13.7492	5.0779	-0.0009
Qwen2-0.5B	0.6453	0.6420	0.6284	0.3228	0.8555	0.0000
Qwen2.5-0.5B	0.6508	0.3721	0.4247	0.3291	1.0530	-0.0000
TinyLlama 1.1B	0.5316	0.0270	0.0168	0.0206	0.0109	-0.0000

Table 5: Full K-only HeadQ frontier. Entries are percent PPL deltas versus dense on four WikiText-103 windows at length 128 with prefill 64.

Model	K	Base	H r2	H r4
Qwen2-0.5B	4	+0.744	+0.113	+0.091
Qwen2-0.5B	3	+3.073	+1.697	+0.389
Qwen2-0.5B	2	+41.510	+11.374	+6.022
Pythia-160M	4	+3.789	+0.846	+0.701
Pythia-160M	3	+15.656	+1.239	+1.260
Pythia-160M	2	+116.894	+9.165	+7.631
GPT-2 124M	4	+0.859	+0.131	+0.099
GPT-2 124M	3	-0.120	+0.798	+0.628
GPT-2 124M	2	+15.630	+2.976	+1.705
Qwen2.5-0.5B	4	+0.110	-0.121	-0.583
Qwen2.5-0.5B	3	+1.775	-0.310	-0.158
Qwen2.5-0.5B	2	+49.135	+9.157	+7.846
TinyLlama 1.1B	4	-0.090	-0.112	-0.234
TinyLlama 1.1B	3	+1.379	+0.677	+0.178
TinyLlama 1.1B	2	+15.797	+12.737	+6.729
Mistral 7B	4	+0.232	+0.256	+0.132
Mistral 7B	3	-0.036	+0.258	-0.090
Mistral 7B	2	+2.140	+2.240	+2.270

Table 6: Same-budget deployed sign control for HeadQ at 3 key bits, rank 4, and 4 side bits. Entries are percent PPL deltas versus dense.

Model	Base	HeadQ	Wrong sign
Qwen2-0.5B	+3.073	+0.278	+1.716
Pythia-160M	+15.656	-0.328	+51.888

Table 7: Full auxiliary KV composition grid. Entries are percent PPL deltas versus dense on four WikiText-103 windows at length 128 with prefill 64.

Model	KV	Base	H + A^2
Qwen2-0.5B	4	+1.955	+0.957
Qwen2-0.5B	3	+5.618	+1.817
Qwen2-0.5B	2	+84.867	+23.108
Pythia-160M	4	+3.730	+0.434
Pythia-160M	3	+25.836	+0.638
Pythia-160M	2	+173.704	+19.795
GPT-2 124M	4	-0.134	-0.678
GPT-2 124M	3	+3.025	+0.709
GPT-2 124M	2	+24.634	+9.072
Qwen2.5-0.5B	4	+1.269	+1.093
Qwen2.5-0.5B	3	-0.181	+0.910
Qwen2.5-0.5B	2	+94.700	+24.498
TinyLlama 1.1B	4	-0.548	-0.073
TinyLlama 1.1B	3	+2.937	+0.211
TinyLlama 1.1B	2	+25.248	+14.562
Mistral 7B	4	+0.328	-0.281
Mistral 7B	3	-1.522	-0.190
Mistral 7B	2	+7.278	+2.507

References

- [1] Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). Language models are unsupervised multitask learners. *OpenAI blog*, 1(8), 9.
- [2] Yang, A., Baheer, B., Chen, J., et al. (2024). Qwen2 technical report. *arXiv preprint arXiv:2407.10671*.
- [3] Hui, B., Yang, J., Cui, Z., et al. (2024). Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*.
- [4] Qwen Team. (2024). Introducing Qwen1.5. Retrieved from <https://qwenlm.github.io/blog/qwen1.5/>.
- [5] Jiang, A. Q., Sablayrolles, A., Mensch, A., et al. (2023). Mistral 7B. *arXiv preprint arXiv:2310.06825*.
- [6] Biderman, S., Schoelkopf, H., Anthony, Q., Bradley, H., O’Brien, K., Hallahan, E., ... & Raff, E. (2023). Pythia: A suite for analyzing large language models across training and scaling. In *International Conference on Machine Learning* (pp. 2397–2430). PMLR.
- [7] Zhang, P., Zeng, G., Wang, T., & Lu, W. (2024). TinyLlama: An open-source small language model. *arXiv preprint arXiv:2401.02385*.
- [8] Merity, S., Xiong, C., Bradbury, J., & Socher, R. (2016). Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843*.
- [9] Liu, Z., Yuan, B., Yin, H., Dong, P., Qin, H., & Yan, J. (2024). KIVI: A tuning-free asymmetric 2bit quantization for KV cache. *arXiv preprint arXiv:2402.02750*.
- [10] Hooper, C., Kim, S., Mohammadzadeh, H., Mahoney, M. W., Shao, Y. S., Keutzer, K., & Gholami, A. (2024). KVQuant: Towards 10 million context length LLM inference with KV cache quantization. *arXiv preprint arXiv:2401.18079*.
- [11] Kang, H., Zhang, Y., Dong, P., ... & Yan, J. (2024). GEAR: An efficient KV cache compression recipe for near-lossless generative inference of large language models. *arXiv preprint arXiv:2403.05527*.
- [12] Zhang, Z., Sheng, Y., Zhou, T., Chen, T., Zheng, L., ... & Re, C. (2023). H2O: Heavy-hitter oracle for efficient generative inference of large language models. *Advances in Neural Information Processing Systems*, 36.
- [13] Ashkboos, S., Ilharco, G., Ahn, M., ... & Hooker, S. (2024). QuaRot: Outlier-free 4-bit inference in rotated LLMs. *arXiv preprint arXiv:2404.00456*.

- [14] Zandieh, A., Daliri, M., Hadian, M., & Mirrokni, V. (2025). TurboQuant: Online vector quantization with near-optimal distortion rate. *arXiv preprint arXiv:2504.19874*.
- [15] Hariri, M., Luo, A., Chen, W., Zhong, S., Zhang, T., Wang, Q., Hu, X., Han, X., Chaudhary, V., et al. (2025). Quantize What Counts: More for Keys, Less for Values. *arXiv preprint arXiv:2502.15075*.
- [16] Tao, Q., Yu, W., & Zhou, J. (2024). AsymKV: Enabling 1-Bit Quantization of KV Cache with Layer-Wise Asymmetric Quantization Configurations. *arXiv preprint arXiv:2410.13212*.
- [17] Li, X., Xing, Z., Li, Y., Qu, L., Zhen, H.-L., Liu, W., Yao, Y., & Pan, S. J. (2025). KVTuner: Sensitivity-Aware Layer-Wise Mixed-Precision KV Cache Quantization for Efficient and Nearly Lossless LLM Inference. *arXiv preprint arXiv:2502.04420*.
- [18] Mao, W., Lin, X., Huang, W., Xie, Y., Fu, T., Zhuang, B., Han, S., & Chen, Y. (2026). TriAttention: Efficient long reasoning with trigonometric KV compression. *arXiv preprint arXiv:2604.04921*.
- [19] Amari, S. (1998). Natural gradient works efficiently in learning. *Neural Computation*, 10(2), 251–276.
- [20] Emadi, S. M. (2026). Exact Attention Sensitivity and the Geometry of Transformer Stability. *arXiv preprint arXiv:2602.18849*.
- [21] Nishida, Y. (2026). AXELRAM: Quantize once, never dequantize. *arXiv preprint arXiv:2604.02638*.
- [22] Su, Z., Chen, Z., Shen, W., Wei, H., Li, L., Yu, H., & Yuan, K. (2025). RotateKV: Accurate and robust 2-bit KV cache quantization for LLMs via outlier-aware adaptive rotations. *arXiv preprint arXiv:2501.16383*.
- [23] Patel, I., & Joshi, I. (2026). PolyKV: A shared asymmetrically-compressed KV cache pool for multi-agent LLM inference. *arXiv preprint arXiv:2604.24971*.
- [24] Karmore, A. (2026). LOOKAT: Lookup-Optimized Key-Attention for Memory-Efficient Transformers. *arXiv preprint arXiv:2601.10155*.
- [25] Liu, Z., Zhao, C., Fedorov, I., Soran, B., Choudhary, D., Krishnamoorthi, R., Chandra, V., Tian, Y., & Blankevoort, T. (2025). SpinQuant: LLM quantization with learned rotations. *International Conference on Learning Representations*.
- [26] Jegou, H., Douze, M., & Schmid, C. (2010). Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(1), 117–128.